SENA · ADSO 228118 · RAP3 · CREAR COMPONENTES FRONTEND

Zustand en React con Vite

RESPUESTAS DE LA GUÍA · PROYECTO: SENAACCESS (PRÁCTICA + TRANSFERENCIA)

Respuestas de la guía

3.1 Actividad de reflexión inicial

1. ¿Qué tiene en común la situación del edificio con la forma en que pasamos props entre componentes de React?

En los dos casos un mensaje atraviesa intermediarios que **no lo necesitan**. En el edificio, la administradora no puede hablar directamente con el quinto piso: el dato pasa por el vigilante, el segundo y el tercero. En React, `App` tiene el carrito y el contador vive en `Navbar`, pero el botón está en `ProductCard`, dentro de `ProductList`, dentro de `CatalogPage`: el dato viaja por todos esos componentes como props, aunque `ProductList` y `CatalogPage` nunca lo usen. Eso es **prop drilling**.

2. ¿Qué pasaría en el edificio si se agregan más pisos o si un intermediario olvida pasar el mensaje? ¿Qué equivale a eso en una aplicación React?

Cada piso nuevo agrega un eslabón y más demora; si alguien se distrae, el mensaje se pierde y el quinto piso nunca se entera. En React pasa lo mismo: agregar un nivel de componentes obliga a modificar todos los intermedios (aunque no usen el dato), y si un componente olvida pasar la prop o la renombra, el componente profundo recibe `undefined`, la UI queda desactualizada y el error aparece lejos de su causa. También empeora el rendimiento: cada cambio re-renderiza a toda la cadena.

3. ¿Qué solución propondría para el edificio? ¿Cómo se traduciría esa solución en una aplicación?

Poner un **citófono o un tablero común**: la administradora publica el aviso y el quinto piso lo lee directamente, sin intermediarios. En React ese "tablero" es un **store global**: con Zustand, `ProductCard` escribe en `useCartStore` y `Navbar` lee del mismo store con un selector. Ninguno pasa props y el árbol deja de importar.

4. En el proyecto de su grupo, ¿qué información necesitan conocer varias pantallas o componentes al mismo tiempo?

1. **Usuario autenticado y token** (navbar, rutas protegidas, peticiones a la API).
2. **Carrito / pedido en curso** (contador en la barra, tarjetas del catálogo, panel de compra).
3. **Notificaciones o mensajes sin leer** (campana, panel de avisos, dashboard).
4. **Tema y preferencias de interfaz** (botón de la barra y toda la aplicación).
5. **Filtros de búsqueda** (barra de filtros y listado de resultados).

3.2 Actividad de contextualización – Parte A (diagnóstico)

1. ¿Qué es el estado (state) de un componente en React y para qué sirve `useState`?

Es la información que un componente recuerda entre renderizados y que, al cambiar, provoca un nuevo render. `useState` es el hook que crea esa variable y su función de actualización: `const [valor, setValor] = useState(inicial)`. Se usa para lo que solo le interesa a ese componente (un input, un modal abierto, un contador).

2. ¿Cuál es la diferencia entre props y estado?

Las **props** las recibe el componente desde su padre y son de solo lectura: el hijo no las modifica. El **estado** es propio del componente, puede cambiar con su `set` y provoca re-render. Las props fluyen hacia abajo; el estado vive donde se declara.

3. ¿Qué sucede con un componente cuando su estado cambia?

React vuelve a ejecutar la función del componente (re-render) y compara el resultado con el render anterior para actualizar solo el DOM necesario. Si el estado se actualiza con una copia nueva, el cambio se detecta; si se muta el mismo objeto, React puede no re-renderizar.

4. ¿Ha usado `useContext` o la Context API? Si es así, describa brevemente para qué.

Sí, por ejemplo para el tema o el idioma: se envuelve la app en un `Provider` y los componentes consumen el valor con `useContext` sin recibirlo por props. Funciona bien con datos que cambian poco, pero **todos los consumidores se re-renderizan** cuando cambia el valor, y hace falta escribir `createContext`, el hook y el provider.

5. ¿Qué es `localStorage` y qué tipo de datos puede guardar?

Es un almacenamiento del navegador que guarda pares clave-valor **en texto** y los conserva al recargar o cerrar la pestaña. Solo guarda strings (objetos hay que serializarlos con `JSON.stringify`), tiene unos 5 MB por origen y es sincrónico. **No** se deben guardar contraseñas ni datos sensibles, porque cualquier script de la página puede leerlo.

6. Escriba el comando para crear un proyecto React con Vite.

```
npm create vite@latest mi-app -- --template react
```

3.3.13 Preguntas de comprobación

1. ¿Qué diferencia hay entre `set({ count: 0 })` y `set((state) => ({ count: state.count + 1 }))`? ¿Cuándo se usa cada uno?

`set({ count: 0 })` escribe un valor **fijo** y hace merge con el estado actual (las demás propiedades se conservan). La forma con función recibe el estado **más reciente** y se usa cuando el nuevo valor **depende del anterior** (`+1`, `-1`, alternar un booleano, agregar a un arreglo). Si se usara `set({ count: count + 1 })` con un `count` capturado del render, dos actualizaciones seguidas podrían perder una.

2. ¿Por qué no se debe escribir `state.items.push(producto)` dentro de una acción?

Porque `push` **muta** el arreglo existente: la referencia sigue siendo la misma, así que los selectores no detectan el cambio y los componentes no se vuelven a renderizar (o lo hacen de forma impredecible). Además `persist` y `devtools` comparan por referencia. Lo correcto es crear un arreglo nuevo: `set((s) => ({ items: [...s.items, producto] }))`.

3. Explique por qué `const store = useCartStore()` puede causar renderizados innecesarios.

Sin selector, el componente se suscribe al **store completo**: se vuelve a renderizar con cualquier cambio (agregar un item, cambiar una cantidad, vaciar el carrito), aunque solo muestre el contador. Con `useCartStore((s) => s.items.length)` se suscribe únicamente a ese valor y solo se re-renderiza cuando ese valor cambia.

4. ¿Para qué sirve `useShallow`?

Para cuando un selector devuelve **un objeto o arreglo** con varios valores. Sin él, el selector crea una referencia nueva en cada render, Zustand 5 la ve como un cambio permanente y puede provocar un ciclo infinito de renderizados. `useShallow` compara el resultado **superficialmente** (clave por clave) y solo notifica si algún valor cambió. En este proyecto se usa en `ProductFilters`, que pide `search`, `category`, `setSearch`, `setCategory` y `clearFilters`.

5. ¿Qué hace el middleware `persist` y dónde guarda la información por defecto?

Guarda el estado del store en el **`localStorage`** del navegador (clave `name`) y lo recupera al iniciar la aplicación, de modo que el estado sobrevive a una recarga. Con `partialize` se elige qué propiedades se guardan (aquí solo `items`, `ids`, `theme`, `user` y `token`). También admite otros almacenamientos (`sessionStorage`, `AsyncStorage` en React Native).

6. Mencione dos datos que deberían ser estado local y dos que deberían ser estado global.

- **Local:** el texto que se escribe en un input de búsqueda antes de aplicarlo; el estado abierto/cerrado de un modal o un menú desplegable.
- **Global:** el usuario autenticado con su token; los items del carrito (o los favoritos, o el tema de la aplicación).

Tienda SENA / SenaAccess · Zustand 5 + React 19 + Vite · Respuestas de las actividades 3.1, 3.2 (Parte A) y 3.3.13.